

Guía de Integración de API en Laravel

Autenticación mediante Sanctum/Tokens y Consumo de Endpoints Protegidos

Esta documentación técnica detalla el procedimiento estándar para realizar la integración y consumo de la API de **Censo Hospitalario** utilizando el cliente HTTP nativo de Laravel. El flujo consta de dos etapas obligatorias:

- **Paso 1: Autenticación (Login):** Envío de credenciales para la obtención del token de acceso temporal (identificado en la respuesta JSON bajo la clave `access_token`).
- **Paso 2: Consumo Protegido (Censo):** Ejecución de peticiones HTTP hacia endpoints restringidos adjuntando el token obtenido en las cabeceras de autorización.

1. Paso 1: Autenticación y Obtención de Token

Para iniciar la comunicación, se debe realizar una petición de tipo `POST` enviando las credenciales asignadas en formato JSON.

Detalles del Endpoint

Propiedad	Valor / Descripción
Método HTTP	<code>POST</code>
URL Completa	<code>https://censo-hospitalario.cacsb.net/api/v1/auth/login</code>
Cabeceras (Headers)	<code>Content-Type: application/json</code> <code>Accept: application/json</code>

Estructura del Cuerpo de la Petición (Payload JSON)

```
{
  "user_name": "soporte.info",
  "password": "xxxxxxxxxxxx"
}
```

Estructura de la Respuesta Exitosa (JSON)

Al autenticarse correctamente, el servidor responderá con un código de estado `200 OK` y un cuerpo JSON que contiene la propiedad `access_token`:

```
{
  "success": true,
  "access_token": "8235|NWlhncpeXIZ0ABLXRQcDkesEeMoF2W0e1NhhUuuS61a370d9",
  "token_type": "Bearer",
}
```

```
"message": "Autenticación exitosa"
}
```

2. Paso 2: Consumo de Endpoint Protegido (Censo Externo)

Una vez almacenado el token de acceso, todas las peticiones posteriores a rutas protegidas deben incluir dicho valor estructurado como un token de tipo **Bearer**.

Detalles del Endpoint

Propiedad	Valor / Descripción
Método HTTP	GET
URL Completa	https://censo-hospitalario.cacsb.net/api/external/censo
Cabeceras (Headers)	Accept: application/json Authorization: Bearer [VALOR_DEL_ACCESS_TOKEN]

Nota Crítica de Seguridad: La cabecera `Accept: application/json` es indispensable. Si el token expira o es inválido, esta cabecera obliga a la API a retornar un error estructurado en formato JSON (código 419 o 401) en lugar de una redirección o error HTML inesperado.

3. Implementación Práctica en Laravel

A continuación, se proporciona un ejemplo de código limpio y estructurado utilizando el wrapper `Illuminate\Support\Facades\Http` incluido en Laravel. Se recomienda encapsular esta lógica dentro de un Servicio dedicado (*Service Pattern*) para mantener una arquitectura mantenible.

Código de Ejemplo: Servicio de Integración

```
<?php

namespace App\Services;

use Illuminate\Support\Facades\Http;
use Illuminate\Support\Facades\Log;
use Exception;

class CensoHospitalarioService
{
    protected string $baseUrl = 'https://censo-hospitalario.cacsb.net/api';
```

```

/**
 * Realiza el flujo completo de autenticación y obtención del censo.
 */
public function obtenerDatosCenso(): ?array
{
    try {
        // 1. Ejecutar autenticación (Paso 1)
        $loginResponse = Http::acceptJson()
            ->post("{ $this->baseUrl}/v1/auth/login", [
                'user_name' => 'soporte.info',
                'password' => 'xxxxxxx', // Reemplazar con credencial real
            ]);

        if ($loginResponse->failed()) {
            Log::error('Fallo en la autenticación del Censo Hospitalario', [
                'status' => $loginResponse->status(),
                'body' => $loginResponse->json()
            ]);
            return null;
        }

        // Extraer token de forma segura
        $token = $loginResponse->json('access_token');

        if (!$token) {
            Log::error('El servicio de login no retornó la clave access_token.');
```

```
        return null;
    }
}
```

Explicación de Métodos Nativos de Laravel Utilizados

- `Http::acceptJson()` : Inyecta automáticamente la cabecera `Accept: application/json` a la llamada saliente.
- `Http::withToken($token)` : Simplifica la sintaxis de autorización, mapeando internamente el string al formato estándar de cabecera `Authorization: Bearer {token}`.
- `$response->json('clave')` : Permite acceder directamente mediante notación de puntos a propiedades internas del JSON de respuesta sin necesidad de decodificar manualmente el string.